

SC102 L12: UI 事件与程序化组件——详细学习笔记

涵盖: C# Event vs Unity Event | UI 事件接口 | PointerEventData | 自定义 UI 组件 | UI 拖动系统

1 C# Event 与 Unity Event

[p.5-6]

C# Event: 基于委托实现的事件系统 (L02 已学)。核心缺点是无法暴露到 Inspector——只能在代码中运行时修改。

Unity Event: 功能与 C# event 相似, 但可以在 Inspector 中直接编辑 (和 Button 的 onClick 一样)。使用时需引入 `UnityEngine.Events` 命名空间。

```
using UnityEngine.Events;
```

```
public UnityEvent myEvent;                // 无参数
public UnityEvent<float> myEvent;          // 泛型版本 (1个float参数)

// 调用
myEvent?.Invoke();                        // ?.是null条件运算符
myEvent.AddListener(SomeFunction);        // 添加响应
```

选择策略: 需要 Inspector 配置 → Unity Event; 只在代码中使用 → C# Event (更轻量, 无 Inspector 序列化开销)。

2 UI 事件接口 (EventSystem Interfaces)

[p.7-8]

Unity 预设了完整的 UI 事件接口, 位于 `UnityEngine.EventSystems` 命名空间。在 IDE 中输入 "IHandler" 可查看所有可用接口:

接口	触发时机
IPointerEnterHandler	鼠标进入 UI 元素
IPointerExitHandler	鼠标离开 UI 元素
IPointerDownHandler	鼠标按下
IPointerUpHandler	鼠标抬起
IPointerClickHandler	鼠标点击
IDragHandler	拖拽中（每帧）
IBeginDragHandler	开始拖拽
IEndDragHandler	结束拖拽
IDropHandler	有物体被放到此 UI 上
ISelectHandler	UI 元素被选中

实现示例：

```
public class MyUI : MonoBehaviour, IPointerEnterHandler {
    public void OnPointerEnter(PointerEventData eventData) {
        // 鼠标进入时的逻辑
    }
}
```

PointerEventData 提供了完整的鼠标输入信息：位置（position）、按下的按钮（button）、点击次数（clickCount）、是否在拖拽（dragging）等。

2.1 UI 事件练习

[p.9]

判断鼠标状态：

```
// (1) 左键是否按下
bool leftPressed = eventData.button == PointerEventData.InputButton.Left;
// (2) 鼠标是否移动
bool moving = eventData.delta != Vector2.zero;
// (3) 鼠标在屏幕左半边
bool leftHalf = eventData.position.x < Screen.width / 2;
// (4) 双击
bool doubleClick = eventData.clickCount == 2;
```

3 自定义 UI 组件

[p.11-14]

Unity 自带 UI 组件有很多限制。自定义组件可以提供更便捷、可控的功能：

常见需求：

1. 防止连接的按钮（冷却时间）
2. 实时进度条
3. 圆形进度条
4. 自带动画效果的按钮（不需 Animator）
5. 支持本地化的文字组件
6. 循环列表（高性能 Scroll View）

自定义 Button 核心思路：继承 Button 基类，覆写 OnPointerDown/OnPointerUp，添加冷却时间逻辑（用协程或计时器控制可点击状态）。

自定义 Slider 核心思路：继承 Slider 基类，覆写 Set 方法，添加实时数值显示/格式化功能。

自定义 Toggle 核心思路：继承 Toggle 基类，覆写 OnValueChanged，添加自定义过渡动画（用 Tweening 而非 Animator）。

4 UI 拖动系统

[p.16-19]

4.1 基础拖动实现

继承三个接口：IBeginDragHandler、IDragHandler、IEndDragHandler。

核心逻辑：

```
public class DraggableUI : MonoBehaviour,
    IBeginDragHandler, IDragHandler, IEndDragHandler {
    private Vector2 offset;

    public void OnBeginDrag(PointerEventData e) {
        offset = (Vector2)transform.position - e.position;
    }
    public void OnDrag(PointerEventData e) {
        transform.position = e.position + offset;
    }
    public void OnEndDrag(PointerEventData e) {
        // 松手后的处理（如检测是否在目标区域上）
    }
}
```

4.2 限制可拖动范围

知道可拖动区域的 RectTransform 后，计算四个边界并用 Mathf.Clamp 限制位置。

4.3 目标响应区域 (Drag Sticker)

当物体被拖入目标区域时，物体被”吸入”（吸附到中心），触发放入事件。通过 OnDrop 检测 + 计算最近目标位置实现。

4.4 生成区域 (Drag Spawner)

允许从某区域”拖出”新物体（而非移动已有物体）。实现：OnBeginDrag 时 Instantiate 预制体 → 初始化拖动状态 → 跟随鼠标。

完整拖动系统流程：Spawner 生成 → Dragger 拖动 → Sticker 吸附 → 事件回调。

5 课后习题详解

[p.20]

5.1 题 1：C# 事件 vs Unity UI 事件

C# Event 优点：轻量（无序列化开销）、类型安全（编译时检查）、纯代码控制（无 Inspector 依赖）、支持多播（一个事件多个订阅者）、与 .NET 生态无缝集成

C# Event 缺点：不可 Inspector 配置、难以非程序员使用、事件源丢失难排查、订阅者生命周期需手动管理（易内存泄漏）、无内置可视化调试

Unity Event 优点：Inspector 可视化、非程序员友好、与 GameObject 生命周期自动关联、支持运行时动态修改回调、内置序列化

Unity Event 缺点：有序列化开销、类型不如 C# 泛型灵活、需引入额外命名空间、仅支持有限参数类型、不适合高频触发事件

5.2 题 2：按右键放弃拖动

```
void Update() {  
    if (Input.GetMouseButtonDown(1) && isDragging) {  
        // 右键按下+正在拖动→放弃拖拽  
        transform.position = dragStartPosition; // 回到原位  
        isDragging = false;  
        OnEndDrag(null); // 手动触发结束  
    }  
}
```

5.3 题 3：实现 ToggleGroup 功能

ToggleGroup 让一组 Toggle 中同时只有一个为选中状态：

```
public class CustomToggleGroup : MonoBehaviour {
    private List<Toggle> toggles = new List<Toggle>();

    public void Register(Toggle toggle) {
        toggles.Add(toggle);
        toggle.onValueChanged.AddListener((isOn) => {
            if (isOn) NotifyOthers(toggle);
        });
    }
    private void NotifyOthers(Toggle activeToggle) {
        foreach (var t in toggles)
            if (t != activeToggle) t.isOn = false;
    }
}
```